

Environments and Deployments in GitLab CI/CD

Managing Review, Staging, and Production Pipelines

Agenda

- Understanding GitLab Environments
- Demystifying Review, Staging, and Production Environments
- Mastering Deployment Control with ``rules`` and ``when:``
- Implementing Manual Approvals in CI/CD Pipelines
- Preparing for Hands-On Labs

Common Environment Types



Review

Isolated, temporary environments for feature branches, enabling rapid code review and validation.



Staging

A replica of production for final integration, performance, and user acceptance testing (UAT).



Production

The live, public-facing environment, serving the end-users.

Common Environment Types: Key Characteristics

Each environment serves a distinct purpose in the software development lifecycle, ensuring quality and stability before reaching the end-user.

Review Environment

Purpose: Feature validation and early feedback for new code.

Nature: Temporary, isolated, automatically provisioned per merge request.

Key Benefit: Speeds up code review by providing a live, testable version of changes.

Staging Environment

Purpose: Comprehensive integration and user acceptance testing (UAT).

Nature: Long-lived, production-like replica, often updated continuously.

Key Benefit: Catches integration issues and performance bottlenecks before production deployment.

Production Environment

Purpose: Delivering the live application to end-users.

Nature: Stable, highly available, public-facing.

Key Benefit: Provides the actual service to customers, requiring robust monitoring and reliability.

Controlling Deployments: `rules` and `when` in Action

GitLab CI/CD's `rules:` and `when:` keywords provide powerful control over job execution, allowing for fine-tuned deployment strategies.



Defining Job Conditions with `rules`

Use `rules:` to specify when a job should run based on conditions like branch names, commit messages, or pipeline variables. This ensures jobs only execute when relevant criteria are met.



Controlling Execution Flow with `when`

The `when:` keyword dictates how a job behaves once its `rules:` are satisfied. Options include `on_success` (default), `manual`, `always`, `delayed`, or `never`, providing flexibility for automated or interactive steps.



Combining for Strategic Pipelines

By combining `rules:` and `when:`, you can create sophisticated workflows. For example, a job might only run on the `main` branch (`rules:`) and require a manual trigger (`when: manual`) for production deployments.

Manual Approvals in Pipelines

Manual approvals provide a critical safeguard for sensitive deployments, ensuring human oversight before critical changes go live.

Preventing Accidents

Adds a human gate, preventing unintended deployments to production or other sensitive environments.

Quality Control

Allows for final review by lead developers or release managers, ensuring adherence to quality standards.

Compliance & Governance

Essential for meeting regulatory requirements and internal governance policies for deployments.

In GitLab CI/CD, this is configured using the `when: manual` keyword in your `.gitlab-ci.yml` file.

Hands-On: Auto-Deploy to Staging

Automate your deployments to the staging environment upon successful merges to the main branch, ensuring a continuously updated pre-production replica.

git

Configure Main Branch Trigger

Set up your CI/CD job to automatically run when changes are merged into the `main` branch, typically using rules: `if: $CI_COMMIT_BRANCH == "main"`.



Define Staging Environment

Utilize the `environment:` keyword with name: `staging` to clearly identify and manage your staging deployments within GitLab.



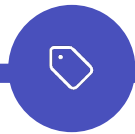
Set Deployment URL

Provide a direct link to your deployed staging application using `environment:url:`, making it easy for testers and stakeholders to access.



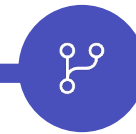
Best Practices for CI/CD Environments

Adhering to these best practices will help you build robust, efficient, and secure CI/CD pipelines for managing your application environments.



Clear Environment Names

Standardize naming conventions for environments (e.g., `review/feature-x`, `staging`, `production`) to ensure clarity and easy identification within GitLab.



Dynamic Preview Environments

Leverage dynamic environments for merge requests to provide isolated, on-demand testing instances for every feature or bug fix, streamlining feedback.



Manual Approvals for Production

Implement mandatory manual approvals for deployments to sensitive environments like production, adding a crucial human gate for quality and security oversight.



Document Environments & Flows

Maintain up-to-date documentation (e.g., in your project's `README.md`) outlining environment purposes, deployment procedures, and access information.

Q&A

Any questions? Let's walk through your GitLab setup!

